Plan d'apprentissage RAG local

De zéro à un pipeline RAG d'entreprise mesuré

Validé en lab, sur CPU puis sur GPU

Architecture : LABO-G9 (Ollama, Windows, GPU) + VM-RAG-LAB (Ubuntu 26.04, Docker Compose)

DoIt4Everyone | Octobre 2026

> *Ce plan documente l'apprentissage progressif d'un pipeline RAG complet, tel qu'il a été construit, mesuré et corrigé en lab. La progression est linéaire : chaque phase s'appuie sur la précédente. Les phases 1 à 12 construisent le pipeline sur un file server Windows et se font entièrement sur CPU. Les phases 13 à 15 ajoutent les connecteurs Microsoft 365 et demandent un tenant actif. Les phases 16 et 17 traitent la preuve du cloisonnement et la mesure, et tirent parti d'une carte graphique. Durée estimée : 25 à 34 semaines à raison de 4 à 6 heures par semaine. Le guide de déploiement publié en donne les procédures détaillées, section par section, avec leur statut de validation.*

1. Architecture cible et prérequis

1.1 Environnement de travail

L'apprentissage se fait dans une configuration qui reproduit ce qu'un client PME aurait en production : le moteur d'inférence sur un poste ou un serveur Windows, le pipeline RAG dans une VM Linux séparée, communication réseau entre les deux.

Composant	Emplacement	Rôle
Ollama (inférence)	LABO-G9, Windows, port 11434	Serveur de modèles : génération de texte et embeddings
qwen3:14b	Ollama sur LABO-G9	Modèle principal de la stack validée (questions et synthèses)
qwen3:4b	Ollama sur LABO-G9	Modèle juge pour le contrôle d'ancrage
nomic-embed-text	Ollama sur LABO-G9	Modèle d'embedding (vectorisation des extraits)
VM-RAG-LAB	VMware Workstation sur LABO-G9	Ubuntu Server 26.04 LTS, stack Docker Compose
Qdrant	Conteneur Docker dans VM-RAG-LAB	Vector store : embeddings, métadonnées et droits d'accès
RAG API FastAPI	Conteneur Docker dans VM-RAG-LAB	Recherche, génération, contrôles et journalisation nLPD
Open WebUI	Conteneur Docker dans VM-RAG-LAB	Interface utilisateur, authentification LDAP AD
n8n	Conteneur Docker dans VM-RAG-LAB	Orchestration : synchronisation, alertes, rappels
mip-service	Conteneur Docker dans VM-RAG-LAB	Déchiffrement des documents protégés par Purview (phase 14)

1.2 Réseau VMware

La VM-RAG-LAB doit atteindre Ollama sur le port 11434 de l'hôte. Deux options :

- VMnet8 (NAT) : la VM accède à l'hôte via l'IP de passerelle NAT. Plus simple, recommandé pour débuter.
- Réseau lab dédié : réseau isolé avec accès aux ressources AD, SMB et DNS. Recommandé à partir de la phase 7.

> **Vérification avant de commencer :** *depuis la VM, curl http://<IP-HOTE>:11434/api/tags doit retourner la liste des modèles. Si la réponse est vide ou si la connexion expire, vérifier qu'Ollama écoute sur le réseau et non sur 127.0.0.1 seulement.*

> **Piège vérifié en lab :** *Ollama tourne dans la session Windows de l'utilisateur. Fermer la fenêtre du Bureau à distance laisse la session ouverte et Ollama actif ; se déconnecter arrête Ollama, et toutes les synchronisations échouent en silence jusqu'à ce qu'on le relance.*

1.3 Sizing de la VM

Ressource	Minimum	Recommandé	Remarque
vCPU	4	6	1 processeur, 6 coeurs dans VMware (évite les problèmes NUMA)
RAM	8 Go	16 Go	8 Go suffisent une fois l'inférence passée sur GPU ; 16 Go sur CPU
Disque	80 Go	100 Go	Type NVMe recommandé ; un second disque chiffré pour les données (§15)
Réseau	NAT	Réseau lab dédié	Requis pour l'accès AD, SMB et DNS à partir de la phase 7

> **Mémoire de l'hôte :** *le GPU ne porte que les modèles ; tout le reste tourne en mémoire système. Sur un hôte qui fait aussi tourner d'autres machines virtuelles, une saturation ne produit aucun message d'erreur : elle se manifeste dans les VM, par du swap et de la lenteur. Surveiller l'hôte autant que la VM.*

1.4 Carte graphique : facultative au départ

Les phases 1 à 16 se font entièrement sur CPU. Les réponses sont lentes, de l'ordre de la minute, mais toutes les notions s'apprennent et tous les contrôles se valident ainsi. C'est même instructif : cette lenteur devient la référence qui permet d'objectiver le gain d'une carte graphique à la phase 17.

La configuration de référence de ce plan utilise une NVIDIA RTX 5060 Ti 16 Go. Une carte de 16 Go permet de charger ensemble le modèle principal, le juge et le modèle d'embedding, avec peu de marge. Une carte plus petite impose de laisser le juge ou l'embedding sur le processeur.

1.5 Première action obligatoire

Avant toute chose, télécharger les trois modèles sur LABO-G9 :

```
ollama pull nomic-embed-text   # embedding, ~274 Mo, tourne aussi sur CPU
ollama pull qwen3:14b          # modèle principal, ~9 Go
ollama pull qwen3:4b           # modèle juge, ~2,5 Go
```

2. Vue d'ensemble des phases

Le plan compte 17 phases, réparties en trois ensembles. Les phases 1 à 12 sont autonomes : elles produisent un pipeline RAG complet et cloisonné sur un file server Windows. Les phases 13 à 15 demandent un tenant Microsoft 365. Les phases 16 et 17 rendent le résultat démontrable et mesurable.

Phase	Sujet	Durée
1	Environnement VM et premier modèle	1 semaine (4-5h)
2	Premier pipeline RAG custom Python	1 semaine (4-5h)
3	Parsing des documents et stratégie de chunking	1-2 semaines (6-8h)
4	Qdrant : persistance, métadonnées et snapshot	1 semaine (4-5h)
5	Prompt strict, citations et contrôle d'ancrage	1-2 semaines (6-8h)
6	Stack Docker Compose complète	1 semaine (4-5h)
7	Permissions NTFS : filtrage par identité AD	2-3 semaines (10-15h)
8	Open WebUI : interface et authentification LDAP	1 semaine (4-5h)
9	Sécurité : injection, durcissement, journalisation nLPD	1-2 semaines (6-8h)
10	Synchronisation et surveillance via n8n	1 semaine (4-5h)
11	Onyx CE : évaluation comparative	1 semaine (4-5h)
12	Projet de synthèse sur corpus réel	2-3 semaines (10-15h)
13 (tenant MS 365)	Connecteur SharePoint Online et permissions Entra ID	2-3 semaines
14 (tenant MS 365)	Documents protégés par Purview	2-3 semaines
15 (tenant MS 365)	Synthèse des réunions Teams	1-2 semaines
16	Contrôle automatisé du cloisonnement	1 semaine (4-5h)
17	Passage sur GPU et méthode de mesure	1-3 semaines

> **Ordre conseillé :** *les phases 16 et 17 sont placées en fin de plan parce qu'elles ont été construites dans cet ordre en lab. Un lecteur qui s'arrête aux phases 1 à 12 a tout intérêt à faire la phase 16 juste après la phase 10 : le contrôle automatisé du cloisonnement vaut pour un corpus SMB seul, et plus tôt il existe, plus tôt il détecte les régressions.*

3. Phases 1 à 12 : le pipeline sur file server Windows

Phase 1 : Environnement VM et premier modèle *1 semaine · 4-5h*

Objectif : avoir une VM opérationnelle, Ollama accessible depuis la VM, comprendre la différence entre un modèle d'embedding et un modèle de langage. Initialiser le dépôt Git qui suivra tous les scripts produits.

Créer la VM-RAG-LAB

- Ubuntu Server 26.04 LTS, 6 vCPU (1 processeur, 6 coeurs), 16 Go RAM, 80 Go NVMe dans VMware Workstation.
- Réseau : VMnet8 (NAT) pour commencer.
- Lors de l'installation : laisser LVM activé, activer OpenSSH, configurer le réseau en statique si la VM rejoint un réseau lab existant.
- Étendre le volume LVM après installation (Ubuntu n'alloue que la moitié du groupe de volumes par défaut) :

```
lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv
```

- Installer Python 3.11, pip, venv, Docker et Docker Compose.
- Initialiser un dépôt Git : git init ~/rag-lab. Committer après chaque phase.

Valider la connectivité Ollama

```
curl http://<IP-HOTE-G9>:11434/api/tags
```

La réponse doit lister les modèles disponibles. Si Ollama n'écoute que sur 127.0.0.1, l'exposer au réseau depuis ses paramètres ou la variable OLLAMA_HOST, puis le redémarrer.

Premier appel depuis la VM

```
curl http://<IP-HOTE>:11434/api/generate \
  -d '{"model":"qwen3:14b","prompt":"Bonjour","stream":false}'
```

Observer le temps de réponse sur CPU. C'est lent, c'est normal : noter cette durée, elle servira de référence à la phase 17.

> **Ce que vous apprenez ici :** *Différence entre un modèle de langage (génère du texte) et un modèle d'embedding (transforme du texte en vecteur). Pourquoi la mémoire est le facteur limitant sur CPU. Pourquoi un pipeline RAG a besoin des deux.*

Phase 2 : Premier pipeline RAG custom Python *1 semaine · 4-5h*

Objectif : indexer un dossier de documents et poser une première question à laquelle le modèle répond à partir des documents, pas de sa mémoire d'entraînement. Observer les limites du découpage par défaut : c'est le point de départ de la phase 3.

Ce plan choisit un pipeline Python écrit à la main plutôt qu'un cadriciel comme LlamaIndex. La raison est pratique : un cadriciel ajoute une couche d'abstraction utile pour découvrir les concepts, mais qui masque le comportement réel du découpage et de la recherche. Le pipeline écrit à la main donne un contrôle total sur chaque étape, indispensable à partir de la phase 5.

Installation de l'environnement Python

```
python3 -m venv .venv && source .venv/bin/activate
pip install qdrant-client httpx python-docx pdfplumber python-pptx
```

Corpus de test recommandé

- Des documents Word (.docx) avec tableaux et listes : ce sont les cas qui cassent le découpage à taille fixe.
- Un PDF textuel et un PDF scanné (image) : pour observer la différence de qualité d'extraction.
- Quelques fichiers .md de documentation technique : pour tester le comportement sur les sections courtes.

> *Inclure au minimum un fichier de chaque type. Les cas qui échouent en phase 2 seront les cas de test de la phase 3.*

Premier script d'indexation

```
# Extraction de texte par type de fichier
#   .docx : python-docx (paragraphes et cellules de tableaux)
#   .pdf  : pdfplumber (page par page, avec détection des scans)
#   .pptx : python-pptx (texte de chaque diapositive)
#   .txt, .md : découpage sur les doubles sauts de ligne
# Embedding via Ollama (nomic-embed-text)
# Stockage dans Qdrant : texte, source, hash MD5
```

> **Ce que vous apprenez ici :** *Le découpage automatique à taille fixe produit souvent des extraits qui coupent un paragraphe ou un tableau en milieu de phrase. La qualité du découpage détermine la qualité maximale atteignable par le RAG, avant même le prompt.*

Phase 3 : Parsing des documents et stratégie de chunking *1-2 semaines · 6-8h*

Objectif : comprendre pourquoi le découpage est la couche qui a le plus d'effet sur la qualité du RAG. Comparer deux approches sur le même corpus et mesurer l'impact sur les questions qui avaient échoué en phase 2.

La stratégie retenue : blocs de paragraphes

La validation en lab a montré qu'une approche simple est plus robuste qu'un découpage sémantique assisté par modèle : découper sur les doubles sauts de ligne, donc sur les paragraphes naturels, avec un seuil minimal de mots pour éviter d'indexer des fragments sans sens.

```
MIN_CHUNK_WORDS = 8     # seuil minimal, configurable
CHUNK_SIZE      = 150   # taille cible en mots
CHUNK_OVERLAP   = 20    # recouvrement entre extraits
```

> *Le seuil de 8 mots est critique pour les fichiers .md à sections courtes : avec un seuil de 15, valeur qui paraît raisonnable, la majorité des sections courtes sont ignorées en silence et le fichier entier produit zéro extrait.*

Cas particuliers validés en lab

- PDF scanné sans couche texte : pdfplumber retourne une page vide. Le fichier part en quarantaine avec un avertissement, pas en erreur silencieuse.
- Fichier .md avec sections courtes (un titre et une phrase) : le seuil de 8 mots permet de les capturer.
- Tableau Word avec cellules fusionnées : extraction cellule par cellule, chaque ligne devient un bloc.

Mesurer l'impact

Sur les questions qui avaient échoué en phase 2 : réindexer le corpus avec le nouveau découpage, reposer les mêmes questions, noter le nombre de bonnes réponses. La progression est la mesure la plus parlante de ce que le découpage apporte. C'est aussi la première fois que le plan impose de mesurer plutôt que de juger à l'œil : cette habitude devient systématique à la phase 17.

> **Ce que vous apprenez ici :** *Pourquoi « un extrait = un paragraphe » est la règle de base, pas le découpage à taille fixe. Pourquoi les tableaux et les listes sont les cas les plus difficiles. Pourquoi un seuil minimal de mots est nécessaire pour les documents à sections courtes.*

Phase 4 : Qdrant, persistance, métadonnées et snapshot *1 semaine · 4-5h*

Objectif : déployer Qdrant en conteneur Docker. L'index survit aux redémarrages. Prendre un premier instantané avant d'aller plus loin.

Lancer Qdrant avec Docker Compose

```
services:
  qdrant:
    image: qdrant/qdrant:latest
    ports: ["6333:6333"]
    volumes: ["./qdrant_data:/qdrant/storage"]
    restart: unless-stopped
```

Métadonnées à stocker pour chaque extrait

Champ	Contenu	Usage
source	Chemin du fichier sur le partage	Citation dans la réponse
chunk_index	Rang de l'extrait dans le document	Reconstituer une fenêtre de lecture (phase 17)
hash_md5	Empreinte du fichier à l'indexation	Détection des modifications
org_name	Organisation déduite de l'arborescence	Navigation et contexte
autorises	Groupes autorisés (ACL NTFS)	Filtrage d'accès (phase 7)
interdits	Groupes refusés (DENY explicites)	Filtrage d'accès (phase 7)
text	Contenu de l'extrait	Injection dans le prompt

> *Les champs autorises et interdits sont vides à ce stade. Ils seront peuplés en phase 7. Les prévoir dès maintenant évite de restructurer le schéma plus tard. Le champ chunk_index paraît anodin : il devient indispensable en phase 17, quand la RAG API doit envoyer une suite d'extraits consécutifs plutôt que des extraits épars.*

Prendre un instantané

```
curl -X POST http://localhost:6333/collections/documents/snapshots
```

Télécharger l'instantané et le conserver hors du conteneur. C'est la sauvegarde de référence pour toutes les phases suivantes.

> **Ce que vous apprenez ici :** *Architecture d'un vector store de production. Différence entre recherche vectorielle (similarité de sens) et filtre de métadonnées (valeur exacte). Les deux seront combinés pour le filtrage par permissions en phase 7.*

Phase 5 : Prompt strict, citations et contrôle d'ancrage *1-2 semaines · 6-8h*

Objectif : construire les garde-fous qui rendent le pipeline fiable. C'est la phase la plus importante pour la fiabilité des réponses, et elle conditionne tout ce qu'on pourra affirmer ensuite devant un décideur.

Prompt système strict

```
Tu es un assistant documentaire. Tu réponds UNIQUEMENT à partir des
documents fournis dans le contexte ci-dessous.
Si la réponse n'est pas dans les documents fournis, réponds exactement :
'Cette information ne figure pas dans les documents disponibles.'
Tu ne complètes jamais avec tes connaissances générales.
Chaque affirmation doit être directement tirée d'un document du contexte.
Cite le nom exact du fichier source entre crochets : [nom_du_fichier.docx]
Ne jamais écrire [Document X] ou numéroter les sources.
/no_think
```

Trois règles ajoutées après mesure

Ces règles ne venaient pas du plan initial : elles répondent à des défauts observés sur des réponses réelles, et leur effet a été mesuré.

- Ne jamais affirmer qu'un document « ne contient pas » une information : le modèle ne voit que des extraits. Il doit écrire que les extraits consultés ne permettent pas de le dire. Une fausse absence est plus insidieuse qu'une invention, parce qu'elle est crédible.
- Chaque citation entre crochets, y compris dans une longue réponse : sans crochets, les citations ne sont ni vérifiables par les contrôles ni transformées en liens.
- Synthétiser sans recopier ni remplir : sur les questions de liste, le modèle recopiait des extraits entiers ou répétait la même formule pour chaque élément.

> **Effet mesuré :** *sur le même jeu de questions, ces règles n'ont pas changé le nombre de bonnes réponses, mais ont divisé par trois la durée des réponses là où le modèle recopiait, et supprimé les fausses absences.*

Contrôles déterministes, sans coût de calcul

- Aucun extrait récupéré : la réponse est bloquée immédiatement.
- Réponse de refus standard : la vérification passe sans appeler le juge.
- Citations inventées : chaque citation entre crochets est comparée aux sources réellement fournies au modèle.
- Réponse longue sans citation : toute réponse substantielle sans citation est bloquée.

> **Ce que le prompt ne suffit pas à obtenir :** *même avec la règle, le modèle continue de citer sans crochets dans les longues réponses, et recopie parfois les balises qui délimitent les documents dans le contexte. Les deux se corrigent côté code, pas côté prompt : la RAG API remet entre crochets les noms des documents réellement fournis, et retire les balises recopiées. Règle générale : ce que le prompt demande sans l'obtenir de façon fiable doit être traité par du code.*

Contrôle d'ancrage : le juge

Un second appel au modèle juge, après la génération, vérifie affirmation par affirmation que la réponse est soutenue par les sources. Le juge reçoit la réponse et l'intégralité des extraits envoyés au modèle principal.

```
JUDGE_MODEL=qwen3:4b
JUDGE_NUM_CTX=      # vide : prend la fenêtre du modèle principal
JUDGE_KEEP_ALIVE=2h # maintien en mémoire entre deux questions
```

> **Règle issue des mesures de la phase 17 :** *la fenêtre de contexte du juge doit couvrir celle du modèle principal. Plus petite, elle fait perdre au juge une partie des sources sans aucun message : il déclare alors non ancrées des affirmations pourtant justes, et le score s'effondre sans que les réponses aient changé.*
>
> **Limite assumée :** *le juge est lui-même un modèle, et il est faillible. En lab, il n'a pas détecté qu'une réponse avait développé un sigle avec les connaissances générales du modèle au lieu du sens donné par le document. Un système de contrôle par modèle réduit le risque, il ne le supprime pas : les cas ratés rejoignent le jeu de questions.*
>
> **Ce que vous apprenez ici :** *Comment le prompt contraint la génération. Pourquoi la température doit être basse pour les tâches factuelles. La distinction entre une réponse fluide et une réponse correcte. Pourquoi certains défauts se corrigent par le code et non par le prompt.*

Phase 6 : Stack Docker Compose complète *1 semaine · 4-5h*

Objectif : déployer la stack complète via Docker Compose. C'est la couche qui transforme des scripts en service opérationnel.

Services déployés

Service	Port	Rôle	Image
Qdrant	6333	Vector store	qdrant/qdrant
n8n	5678	Orchestration des pipelines	n8nio/n8n
RAG API	8080	FastAPI : recherche, génération, synchronisation	Build local (api/)
Open WebUI	3001	Interface utilisateur, authentification LDAP	ghcr.io/open-webui/open-webui

> **Versions figées :** *le Compose du lab épingle une version précise de chaque image, avec son empreinte. En lab, l'image flottante d'Open WebUI et la version étiquetée du même numéro ne contenaient pas le même code, et passer de l'une à l'autre a révélé un défaut latent de la RAG API : plus aucune réponse ne s'affichait alors que l'API répondait correctement. Une version flottante masque ce genre de dépendance jusqu'au jour où elle casse.*

Variables d'environnement clés

```
LLM_BASE_URL=http://<IP-HOTE-OLLAMA>:11434
LLM_MODEL=qwen3:14b
LLM_NUM_CTX=16384
JUDGE_MODEL=qwen3:4b
```

```
EMBED_MODEL=nomic-embed-text
QDRANT_HOST=http://qdrant:6333
MIN_CHUNK_WORDS=8
TOP_K=20
```

> **Piège vérifié deux fois en lab :** *une variable présente dans le fichier .env n'atteint le conteneur que si elle figure aussi dans la section environment du service. Une variable oubliée là reste sans effet, sans aucun message : deux demi-journées de mesures ont été perdues ainsi. Toujours vérifier un réglage dans le conteneur, jamais seulement dans le .env. Et une variable transmise vide n'est pas une variable absente : un réglage numérique lu comme vide empêche le service de démarrer.*

Structure des répertoires

```
~/rag-stack/
  ├── .env                  <- variables sensibles, jamais dans Git
  ├── docker-compose.yml
  ├── qdrant_data/          <- données Qdrant
  ├── n8n_data/             <- workflows n8n
  ├── openwebui_data/       <- données Open WebUI
  └── api/
      ├── Dockerfile
      ├── main.py           <- RAG API complète
      └── auth.py           <- résolution des groupes
/root/rag-pipeline/         <- scripts d'indexation (montés en lecture
seule)
```

> **Ce que vous apprenez ici :** *Comment un pipeline RAG devient un service. La différence entre un réglage défini et un réglage réellement appliqué. Pourquoi figer les versions d'images est une décision de fiabilité, pas de pédanterie.*

Phase 7 : Permissions NTFS, filtrage par identité AD *2-3 semaines · 10-15h*

Objectif : propager les permissions NTFS jusqu'aux extraits dans Qdrant et filtrer les résultats selon l'identité de l'utilisateur. C'est la phase la plus différenciante vis-à-vis d'un intégrateur standard.

> **Préparer avant de démarrer :** *deux comptes AD avec des droits différents sur au moins deux fichiers du corpus. Les groupes imbriqués et les refus explicites prendront plus de temps que prévu. Prévoir 10 à 15 heures, pas 6 à 8.*

Lecture des ACL NTFS depuis Linux

```
# Monter le partage SMB (persistant via fstab)
//<FILESERVER>/PartageDocuments /mnt/corpus-root cifs \
  credentials=/etc/smbcredentials/svc-rag,vers=3.1.1 0 0

# Lire les ACL
smbcacls //<FILESERVER>/PartageDocuments fichier.docx -U svc-rag
```

Le masque des droits : la faille à ne pas rater

Une entrée ACL ne dit pas seulement qui est autorisé, elle dit à quoi. Une première version du résolveur retenait toute entrée de type ALLOW sans lire son masque de droits. Conséquence trouvée en lab : un dossier de dépôt, où un groupe large peut déposer des fichiers sans les lire, devenait interrogeable par tous ceux qui pouvaient y écrire.

```
# smbcacls affiche le masque par son nom (READ, CHANGE, FULL)
# ou en hexadécimal. Exemple d'un droit d'écriture seule :
#   0x00100116   -> écriture, sans lecture
```

```
# Seules les entrées qui accordent la LECTURE donnent accès au contenu.
```

> **Leçon d'ordre :** *corriger la lecture du masque avant de s'attaquer aux cas plus rares comme le groupe principal d'un compte. Sans cela, un fichier en écriture seule pour les utilisateurs du domaine devient lisible par tout le domaine.*

Résolution récursive des groupes

Un utilisateur peut être membre d'un groupe qui est lui-même membre d'un autre groupe. La résolution doit être récursive, avec un cache court pour éviter un appel à l'annuaire à chaque question.

Filtrage à la question

```
# Filtre Qdrant : intersection des groupes de l'utilisateur
# avec autorises[], exclusion si intersection avec interdits[]
# Le refus explicite l'emporte toujours sur l'autorisation.
```

Tests obligatoires

- Cloisonnement positif : un utilisateur d'un groupe métier voit les documents de ce groupe.
- Cloisonnement négatif : ce même utilisateur ne voit pas les documents d'un autre service.
- Refus explicite : un utilisateur avec un DENY nominatif ne voit pas le fichier, même si son groupe est autorisé.
- Groupes imbriqués : un utilisateur d'un sous-groupe voit ce que le groupe parent autorise.
- Écriture seule : un dossier de dépôt n'est pas interrogeable par ceux qui ne peuvent qu'y écrire.

> *Le refus explicite et l'écriture seule sont les deux cas que la majorité des implémentations ratent. Les tester explicitement avant de passer à la phase suivante. La phase 16 transformera ces tests manuels en contrôle automatique.*
>
> **Ce que vous apprenez ici :** *Pourquoi les refus l'emportent sur les autorisations dans Windows. Pourquoi une entrée ALLOW ne suffit pas : c'est son masque qui dit s'il y a lecture. La différence entre indexer avec un compte de service et servir avec l'identité de l'utilisateur. Pourquoi la resynchronisation des droits est un travail distinct de celle des contenus.*

Phase 8 : Open WebUI, interface et authentification LDAP *1 semaine · 4-5h*

Objectif : déployer Open WebUI comme interface finale, avec authentification LDAP Active Directory et transmission de l'identité à la RAG API pour le filtrage documentaire.

Configuration LDAP

Paramètre	Valeur	Note
Port	636	LDAPS obligatoire, le port 389 est refusé par les contrôleurs récents
Attribut email	userPrincipalName	L'attribut mail est souvent non renseigné dans l'AD
Attribut utilisateur	sAMAccountName	

Paramètre	Valeur	Note
Base de recherche	DC=domaine,DC=ch	Pas une seule unité d'organisation : des comptes vivent ailleurs
Validation du certificat	Activée	Exige le bundle de l'autorité exporté depuis le contrôleur

Transmission de l'identité

Sans la variable qui active le relais des informations utilisateur, Open WebUI traite la RAG API comme un simple point d'accès compatible OpenAI et ne transmet aucune identité : le cloisonnement ne peut pas fonctionner.

```
ENABLE_FORWARD_USER_INFO_HEADERS=true

# En-têtes reçus par la RAG API :
x-openwebui-user-email   <- sert à la résolution des groupes
x-openwebui-user-name
x-openwebui-user-id
```

Validation de bout en bout

- Tester avec deux comptes aux droits différents : chacun ne doit voir que son périmètre.
- Vérifier dans les journaux de la RAG API que l'identité et les groupes sont correctement résolus.

> **Panne à prévoir dans le code :** *si la résolution de l'identité échoue, par exemple sur une coupure réseau passagère, l'utilisateur se retrouve avec moins de groupes que la réalité et reçoit un refus trompeur « cette information ne figure pas dans les documents ». La RAG API doit distinguer ce cas et le dire, plutôt que de laisser croire à une absence.*
>
> **Ce que vous apprenez ici :** *Comment une interface transmet l'identité à une API tierce. Pourquoi userPrincipalName est préférable à mail dans un AD Windows. La chaîne complète : navigateur, interface, RAG API, Qdrant, modèle.*

Phase 9 : Sécurité, injection, durcissement, journalisation nLPD *1-2 semaines · 6-8h*

Objectif : tester les vecteurs d'attaque propres au pipeline RAG et mettre en place les contre-mesures.

> *La sécurité du pipeline RAG est l'angle mort de la quasi-totalité des offres d'intégrateurs. La tester soi-même est le seul moyen d'en parler avec autorité.*

Test 1 : injection par un document indexé

Créer un PDF contenant du texte invisible, de la même couleur que le fond, avec une instruction hostile. L'indexer, poser ensuite une question normale, et observer si le modèle suit l'instruction injectée.

Mitigation : séparation structurelle entre le prompt système, fixe, et les extraits, placés dans une zone explicitement marquée comme des données. Vérifier que l'injection échoue après mitigation.

Test 2 : accès sans authentification

Interroger la RAG API sans jeton. Si le service répond, le pipeline est exposé sans contrôle d'accès.

Test 3 : compte de service désactivé

> **Piège validé en lab :** *si le compte de service est désactivé alors que la synchronisation tourne, les extraits sont réindexés sans droits et plus personne ne voit rien. Ne jamais désactiver le compte de service sans couper la synchronisation au préalable.*

Secrets restés à leur valeur d'exemple

Deux jetons du lab étaient restés à « changeme » pendant des semaines, et la stack fonctionnait sans rien signaler. La RAG API avertit maintenant au démarrage quand un secret a gardé sa valeur d'exemple. Un garde-fou de ce genre coûte dix lignes et trouve une vraie faille.

Journalisation nLPD

```
{
  "timestamp": "2026-09-15T09:20:48Z",
  "user_id": "utilisateur@domaine.ch",
  "question_hash": "a394957914563e32",
  "sources_accessed": ["CLIENTS/contrat.docx"],
  "ancree": true,
  "verification": "effectuee"
}
```

La question est empreinte, pas stockée en clair. On peut démontrer qu'une question a été posée et quels documents ont été consultés, sans exposer le contenu des échanges.

Pare-feu et TLS

- Pare-feu par défaut en refus, ouverture des seuls ports nécessaires depuis le sous-réseau du lab.
- Attention : Docker contourne le pare-feu de l'hôte en publiant ses ports. Le vérifier depuis une autre machine, ne pas se fier aux règles seules.
- LDAPS avec validation du certificat, en utilisant le nom DNS du contrôleur et jamais son adresse IP.

> **Ce que vous apprenez ici :** *Comment un document anodin peut compromettre un pipeline RAG non protégé. Pourquoi la séparation structurelle instructions et données est la mitigation principale. Pourquoi la journalisation est une exigence, pas une option. Pourquoi une configuration qui fonctionne n'est pas une configuration sûre.*

Phase 10 : Synchronisation et surveillance via n8n *1 semaine · 4-5h*

Objectif : automatiser la synchronisation du corpus et la résolution des droits. Un RAG non maintenu se dégrade sur trois vecteurs : contenu périmé, permissions obsolètes, dérive du modèle.

Un point d'entrée unique

La RAG API expose un point d'entrée de synchronisation qui enchaîne l'indexation puis la résolution des droits, et renvoie un rapport. n8n ne fait que le déclencher et traiter le résultat.

```
curl -X POST http://localhost:8080/admin/sync \
  -H "Authorization: Bearer <JETON_ADMIN>"
```

Pipeline : synchronisation horaire

- Déclencheur planifié, toutes les heures.
- Appel du point d'entrée de synchronisation, avec le jeton admin rangé dans un identifiant n8n, pas en clair dans le workflow.
- Si la synchronisation échoue ou met des fichiers en quarantaine, envoi d'une alerte par courriel détaillant la cause.

Pipeline : rappel de rotation du compte de service

- Déclencheur planifié mensuel, courriel de rappel. La rotation reste manuelle.

> *La resynchronisation des droits est un travail distinct de celle des contenus. Un fichier peut n'avoir pas changé alors que ses permissions ont été modifiées.*
>
> **Ce que vous apprenez ici :** *Les trois vecteurs de dégradation d'un RAG. Pourquoi les droits se resynchronisent séparément des contenus. Pourquoi une erreur de synchronisation doit produire un message explicite : un rapport qui dit seulement « code 1 » oblige à chercher la cause à la main.*

Phase 11 : Onyx CE, évaluation comparative *1 semaine · 4-5h*

Objectif : déployer une plateforme RAG packagée et comprendre ce qu'elle apporte par rapport au pipeline construit dans les phases précédentes. Identifier ses limites.

```
git clone https://github.com/onyx-dot-app/onyx.git
cd onyx/deployment/docker_compose
docker compose -f docker-compose.dev.yml up -d
```

OpenSearch réclame 2 Go de mémoire par défaut : le limiter pour un lab. La variable qui le règle n'est pas lue depuis le fichier .env, elle doit être ajoutée dans la section environment du service concerné.

Comparaison sur le même corpus

Critère	Pipeline construit aux phases 1 à 10	Plateforme packagée
Qualité des réponses	Mesurée sur un jeu de questions	À mesurer sur le même jeu
Cloisonnement NTFS	Validé, refus explicites compris	Non natif
Contrôle d'ancrage	Juge dédié, règles d'attribution	Non inclus
Journalisation nLPD	Structurée, question empreinte	Non incluse
Maintenance	Scripts Python et Docker Compose	Stack complexe à plusieurs composants
Connecteurs	Les formats et les sources construits au besoin	Catalogue étendu

> **Nettoyage à ne pas oublier :** *une plateforme abandonnée laisse ses volumes Docker derrière elle. En lab, dix volumes contenant une copie complète du corpus sont restés en place après l'abandon d'Onyx, hors du cloisonnement et hors du chiffrement. Un point de gouvernance à part entière.*

> **Ce que vous apprenez ici :** *Valeur et limites d'une plateforme packagée. Ce que représente la maintenance d'une stack open source. Comment évaluer objectivement une offre d'intégrateur qui présente une plateforme comme une solution clé en main.*

Phase 12 : Projet de synthèse sur corpus réel *2-3 semaines · 10-15h*

Objectif : construire un RAG complet sur un cas d'usage réel. C'est la phase qui transforme l'apprentissage en compétence démontrable.

Cas d'usage recommandé

Une base de connaissances technique interne : des guides, procédures et documentations sur un domaine maîtrisé constituent un corpus de qualité, dont les bonnes réponses sont connues d'avance. C'est ce qui permet d'évaluer le système sans ambiguïté.

Livrable du projet

- Pipeline complet en conteneurs : vector store, RAG API, interface, orchestration, synchronisation, droits, journalisation.
- Stratégie de découpage documentée et mesurée sur le corpus réel.
- Rapport de déploiement : choix techniques, modèles, quantification, résultats mesurés.
- Jeu de 30 questions de test avec réponses attendues et résultats obtenus.
- Documentation de maintenance : synchronisation, surveillance de la dérive, procédure de restauration.
- Test de sécurité documenté : injection testée et mitigée, authentification vérifiée, journalisation validée.

> **Critère de réussite du plan :** *déployer, configurer, sécuriser, évaluer et maintenir un pipeline RAG complet, expliquer chaque choix technique à un décideur non technicien, et identifier les failles d'une offre d'intégrateur. Les douze premières phases couvrent les trois.*
>
> **Ce que vous apprenez ici :** *Ce qui sépare un prototype qui répond bien d'un service qu'on peut confier à des utilisateurs : la maintenance, la mesure et la preuve.*

4. Phases 13 à 15 : connecteurs Microsoft 365

Ces trois phases demandent un tenant Microsoft 365 actif et des droits d'administration sur Entra ID. Elles sont séparées du reste parce que ce prérequis n'est pas garanti dans un lab personnel, et parce qu'elles n'apportent rien à une organisation dont les documents vivent uniquement sur un file server.

> **Souveraineté :** *si le tenant Microsoft 365 n'est pas hébergé en Suisse, les requêtes d'indexation transitent hors juridiction suisse, même si le modèle tourne en local. La souveraineté des données dépend du datacenter du tenant, pas du moteur d'inférence.*
>
> **Principe commun aux trois phases :** *une application distincte par usage, avec son propre certificat et le minimum de droits. Quatre applications ont été créées en lab : résoudre les identités, indexer SharePoint, déchiffrer les documents protégés, lire les transcriptions Teams. Une application unique aurait cumulé des droits qu'aucun usage ne justifie à lui seul, et personne n'aurait pu dire laquelle servait à quoi.*

Phase 13 : Connecteur SharePoint Online et permissions Entra ID *2-3 semaines · tenant MS 365, droits admin Entra ID*

Objectif : indexer les bibliothèques SharePoint en traduisant leurs permissions jusqu'aux extraits, comme la phase 7 l'a fait pour les ACL NTFS.

Inscription d'application

- Authentification par certificat, jamais par secret client.
- Permission Sites.Selected, de type application. Ne jamais utiliser Sites.Read.All, qui expose tout le tenant.
- Sites.Selected ne donne rien par elle-même : l'accès se concède site par site, explicitement. C'est précisément ce qui en fait la bonne permission.

Traduire les permissions SharePoint

C'est la partie longue. Les droits d'un document SharePoint peuvent venir de sources très différentes, et chacune se présente autrement :

- Groupes SharePoint du site (propriétaires, membres, visiteurs), lisibles par l'API REST du site.
- Groupes Microsoft 365 et groupes de sécurité synchronisés depuis l'AD, identifiés par des revendications de formes distinctes.
- Permissions accordées directement à une personne.
- Liens de partage : à l'organisation, à des personnes nommées. Un lien anonyme doit faire exclure le document.
- Une revendication « tous les utilisateurs internes » sur le site racine, qui n'est pas un groupe de l'annuaire.

> **Identifiants introuvables :** *un site peut conserver la référence d'un groupe supprimé depuis longtemps, ou d'un rôle d'annuaire qui n'est pas un groupe. Ces entrées doivent être ignorées explicitement et signalées dans le rapport, jamais traitées comme une autorisation.*

Indexation incrémentale

- Le contenu n'est retéléchargé que si l'empreinte de version du fichier a changé.
- Les permissions, elles, sont recalculées à chaque passage : elles changent sans que le fichier change.

- Exclure les bibliothèques système, en particulier celle qui conserve les anciennes versions pour la rétention : elle contient des copies de documents retirés.
- Les fichiers chiffrés par une étiquette de sensibilité sont détectés, comptés dans le rapport et ignorés. Leur traitement est l'objet de la phase 14.

Matrice de validation

Rejouer la matrice de la phase 7 sur le corpus SharePoint, avec au moins trois comptes aux droits différents, dont un compte sans licence pour vérifier le cas de l'accès large.

> **Ce que vous apprenez ici :** *La différence entre une permission qui expose tout un tenant et une permission concédée site par site. Comment des droits cloud se traduisent dans un index local. Pourquoi une permission inconnue doit toujours être traitée comme un refus.*

Phase 14 : Documents protégés par Purview *2-3 semaines · tenant MS 365, droits admin*

Objectif : rendre interrogeables des documents chiffrés par une étiquette de sensibilité, sans affaiblir la protection qu'ils portent. C'est la phase la plus délicate du plan, autant sur le plan technique que sur celui de la gouvernance.

> **À lire avant de commencer :** *la vraie question n'est pas « peut-on indexer des documents chiffrés », mais « faut-il le faire ». Un document déchiffré pour être indexé voit son contenu en clair entrer dans le périmètre de sécurité du serveur : ses disques, ses sauvegardes et ses journaux deviennent aussi sensibles que le document d'origine. Pour des données RH, la recommandation issue du lab est simple : ne pas les indexer.*

Architecture retenue

- Un service de déchiffrement séparé, dans son propre conteneur, sans port publié : seule la RAG API le joint, par le réseau interne.
- Le déchiffrement se fait en mémoire ; le fichier en clair ne touche pas le disque.
- Le service refuse de lui-même certains cas avant tout déchiffrement : étiquette exclue par configuration, protection définie par l'utilisateur, autorisation expirée.

> **Piste abandonnée :** *la clé de protection n'est pas récupérable depuis un coffre de clés, qui sert à ranger le certificat de l'application et non la clé du document. Le déchiffrement passe obligatoirement par le kit de développement de Microsoft Information Protection, qui existe en version Linux.*

La double condition à la question

Avoir accès au fichier dans SharePoint ne suffit pas : l'étiquette porte ses propres droits. À chaque question, l'accès exige les deux conditions, et les droits de l'étiquette sont évalués pour l'utilisateur qui pose la question, pas pour le compte d'indexation.

```
Accès accordé si :
   permissions SharePoint de l'utilisateur   (phase 13)
ET droits de l'étiquette pour cet utilisateur (évalués à la question)

Pour un contenu chiffré, le principe est celui de Copilot :
   propriétaire, ou droits de consultation ET d'extraction.
```

Un constat de gouvernance

> **Vérifié en lab :** *désactiver la fonction super-utilisateur du tenant ne suffit pas. Une application à qui le droit de déchiffrement a été accordé continue de l'exercer. Les*

> *autorisations des applications capables d'extraire du contenu protégé doivent donc être auditées séparément, comme un inventaire à part entière.*

Conséquences obligatoires

- Clé d'API sur le vector store : sans elle, l'index est interrogeable sans aucun contrôle.
- Chiffrement du disque de données au repos, puisque le texte en clair y est stocké.
- Une liste explicite des étiquettes que l'on accepte d'indexer, et une liste de celles qu'on n'indexera jamais.
- Une matrice de validation avec des comptes couvrant les quatre cas : accès complet, droits d'étiquette sans accès au site, accès au site sans droits d'étiquette, aucun des deux.

> **Ce que vous apprenez ici :** *Pourquoi la capacité technique ne décide pas de l'usage. Ce que signifie réellement « déchiffrer pour indexer » du point de vue du périmètre de sécurité. Pourquoi une autorisation accordée à une application survit à la désactivation d'une fonction du tenant.*

Phase 15 : Synthèse des réunions Teams

1-2 semaines · tenant MS 365 avec Teams, droits admin

Objectif : récupérer les transcriptions de réunions, produire un compte rendu structuré avec le modèle local, et le remettre à la bonne personne.

Prérequis d'administration, à faire en premier

- Activer la transcription dans la stratégie de réunion, et attendre sa propagation, jusqu'à 24 heures selon le tenant.
- Activer explicitement l'accès des applications aux transcriptions : ce réglage est désactivé par défaut et l'API renvoie sinon un refus, sans rapport apparent avec la permission accordée.
- Activer l'attribution des intervenants, sinon la transcription arrive sans savoir qui parle. Microsoft avertit à juste titre que cette attribution peut exposer des informations personnelles.
- Une stratégie d'accès applicatif, attribuée non pas utilisateur par utilisateur, ce qui n'est pas tenable en entreprise, mais à un groupe de sécurité auquel les organisateurs adhèrent volontairement. L'adhésion au groupe devient le consentement.

Destination du compte rendu

> **Décision du lab :** *le compte rendu n'est pas publié automatiquement dans un canal. Il part en brouillon par courriel à l'organisateur de la réunion, qui le relit et le diffuse s'il le juge bon. Un compte rendu généré par un modèle et publié sans relecture engage l'organisation sur un texte que personne n'a validé. Cette décision supprime au passage le besoin d'une permission d'écriture dans Teams.*

Évaluer la synthèse, pas la regarder

Une synthèse se juge sur une grille, pas à l'impression. Le lab utilise une transcription de test et un corrigé : décisions, actions avec leur responsable et leur échéance, points ouverts, et des pièges délibérés dont une donnée de santé qui ne doit pas figurer dans le compte rendu.

> **Leçon de méthode :** *quatre versions du prompt ont été comparées. La version qui semblait la meilleure inventait un fait absent de la transcription, et le contrôle automatique par mots-clés ne l'avait pas vu. La version retenue est celle qui n'invente*

rien, même avec un score brut légèrement inférieur. Un passage unique ne suffit jamais à départager deux prompts : la variation d'un passage à l'autre est du même ordre que l'écart mesuré.

Ce que vous apprenez ici : *Comment une API expose des réunions et leurs transcriptions. Pourquoi l'activation côté administration est un prérequis non négociable, et souvent invisible. Pourquoi la destination d'un compte rendu est une décision de gouvernance avant d'être un choix technique.*

5. Phases 16 et 17 : preuve et mesure

Les quinze premières phases produisent un pipeline qui fonctionne. Les deux dernières le rendent démontrable : la phase 16 transforme le cloisonnement en preuve rejouable, la phase 17 installe une méthode de mesure qui permet de décider par les chiffres plutôt que par l'intuition.

Phase 16 : Contrôle automatisé du cloisonnement *1 semaine · 4-5h*

Objectif : passer d'une matrice de tests faite à la main à un contrôle rejoué automatiquement après chaque synchronisation.

Pourquoi la matrice manuelle ne suffit pas

Une matrice de tests prouve que le cloisonnement fonctionnait le jour du test. Elle ne dit rien du lendemain, après une réindexation, un changement de permissions ou une modification du pipeline. Le contrôle doit donc être automatique et se rejouer tout seul.

Construire le jeu de cas

Chaque cas associe une question, un document, un secret attendu, et le verdict attendu pour chaque compte de test. Un bon jeu couvre au minimum :

- Un refus explicite sur un fichier dont le groupe de l'utilisateur est par ailleurs autorisé.
- Un dossier de dépôt en écriture seule.
- Un document protégé par une étiquette, pour un compte qui a le site mais pas les droits de l'étiquette.
- Un document accessible à tous, pour vérifier qu'on ne refuse pas à tort.

Deux modes de contrôle

- Mode accès : vérifie quels extraits sont visibles pour chaque compte. Rapide, il tourne après chaque synchronisation.
- Mode complet : pose réellement la question et vérifie que le document est absent des sources et le secret absent de la réponse. Plus lent, il valide le chemin de bout en bout.

Trois exigences apprises en lab

- Un cas doit désigner un document exact. Deux documents de même nom dans deux emplacements différents donnent un cas ambigu, qui réussit grâce à l'un sans qu'on sache lequel. Un nom partagé doit être refusé comme non probant.
- Une panne doit produire une erreur, jamais un verdict. Une erreur de lecture du vector store traitée comme une collection absente transforme une panne en succès apparent.
- Le contrôle doit alerter. En lab, il s'exécute à la fin de chaque synchronisation et déclenche un courriel si un seul cas échoue.

> **Ce que le contrôle a trouvé :** *dès sa mise en place, il a révélé une faille réelle que la matrice manuelle avait laissée passer, celle du dossier de dépôt en écriture seule décrite en phase 7. Plus tard, il a aussi révélé qu'un cas de test portait sur deux exemplaires d'un même document. Un mécanisme de preuve qui ne trouve jamais rien n'a probablement jamais été mis à l'épreuve.*

> **Ce que vous apprenez ici :** *La différence entre un test passé une fois et une preuve rejouable. Pourquoi un contrôle de sécurité doit échouer bruyamment plutôt que réussir silencieusement. Pourquoi la précision d'un cas de test fait la valeur de la preuve.*

Phase 17 : Passage sur GPU et méthode de mesure *1-3 semaines*

Objectif : installer une carte graphique, mesurer ce qu'elle change, et surtout se doter d'une méthode de mesure reproductible. C'est cette méthode, plus que la carte, qui fait la valeur de la phase.

Installer la carte

- Vérifier l'alimentation et le connecteur avant d'acheter, et placer la carte dans l'emplacement réellement câblé en pleine largeur.
- Installer le pilote seul, en installation personnalisée. La boîte à outils de développement, plusieurs gigaoctets, est inutile : le moteur d'inférence embarque ses propres bibliothèques.
- Régler le moteur d'inférence pour une attention économe en mémoire, un cache de contexte sur 8 bits, une requête à la fois, et un maintien des modèles en mémoire.

> **Vérifier le résultat, pas le réglage :** *deux réglages définis dans les variables d'environnement se sont révélés ignorés en lab. Le journal du moteur d'inférence affiche au démarrage la configuration réellement lue et le moteur de calcul retenu : c'est cela qu'il faut lire.*

Placer les modèles

Sur une carte de 16 Go, les trois modèles tiennent ensemble, avec environ un gigaoctet de marge. Le placement se règle modèle par modèle, et la commande d'état du moteur d'inférence doit afficher chacun entièrement sur la carte. Un modèle partagé entre carte et processeur signale qu'il faut réduire la fenêtre de contexte.

> **Décision renversée par la mesure :** *le plan initial prévoyait de laisser le juge sur le processeur, pour réserver la carte au modèle principal. Mesuré, le juge devenait le goulet d'étranglement : une question vérifiée prenait encore une minute. Placé sur la carte, il répond en moins de deux secondes. La mesure a renversé une décision qui paraissait pourtant raisonnable.*

La méthode de mesure

C'est le vrai livrable de cette phase. Elle tient en quatre éléments :

- Un jeu de questions fixe, avec pour chacune des critères vérifiables automatiquement : le document attendu en tête du contexte, les éléments attendus dans la réponse, les éléments interdits, le refus attendu ou non.
- Une graine de génération fixée pendant les campagnes, qui rend les réponses reproductibles et permet de comparer deux configurations à conditions égales. Un seul passage suffit alors. En usage normal, la graine reste libre.
- Un script qui pose les questions comme le ferait un utilisateur, évalue chaque réponse et produit un rapport daté, avec la configuration mesurée.
- Une règle : une variable à la fois. Deux changements simultanés rendent le résultat ininterprétable.

> **Piège du cache :** *le moteur d'inférence conserve le calcul du dernier texte lu. Une question posée deux fois de suite est servie beaucoup plus vite : on mesure alors le cache, pas le modèle. Poser les questions dans l'ordre, passage après passage.*

Ce que le GPU change, et ce qu'il ne change pas

Mesure	CPU	GPU
Question précise vérifiée par le juge	1 min 15 à 1 min 30	2 à 6 s
Question large	1 min 30 à 2 min	7 à 20 s
Synthèse d'une réunion de test	100 à 146 s	11 s
Qualité mesurée sur le jeu de questions	Référence	Inchangée
Contrôle du cloisonnement	Tous les cas conformes	Tous les cas conformes

Le gain est d'un facteur cinq à dix sur les temps de réponse, sans aucun effet sur la qualité ni sur la sécurité. C'est attendu : ce sont les mêmes modèles. Ce que la carte change vraiment, c'est la possibilité de mesurer : une campagne de quinze questions prend une à deux minutes au lieu d'une demi-heure, et les comparaisons qui suivent deviennent réalistes.

Ce qui améliore vraiment la qualité

Les comparaisons menées avec cette méthode ont donné des résultats contre-intuitifs, et c'est leur intérêt :

- Changer de modèle d'embedding pour un modèle réputé meilleur en français n'a rien changé sur un corpus francophone : match nul, pour un modèle trois fois plus lourd.
- Ajouter un reclasseur, étape que presque tous les tutoriels recommandent, a fait baisser le score et coûté cinq secondes par question. Il ignore les noms propres, ce qui lui fait confondre deux contrats au contenu voisin. Écarté, en laissant l'option disponible.
- En revanche, la construction du contexte a tout changé sur les questions larges : envoyer un document court en entier plutôt qu'une fenêtre tronquée, et donner à chaque document complémentaire une suite d'extraits consécutifs plutôt que des extraits épars.
- Et les règles de prompt de la phase 5 ont divisé par trois la durée des réponses sur les questions de liste.

> **Conclusion de la phase :** *le matériel apporte la vitesse. La qualité se gagne dans la construction du contexte, dans le prompt, et surtout dans les documents eux-mêmes. Les limites qui restent après toutes ces mesures ne se corrigent par aucun réglage : questions d'inventaire sur un corpus sans page de synthèse, deux versions contradictoires d'un même contrat, documentation du système indexée dans le système. Ce sont des sujets de gouvernance documentaire, pas de paramétrage.*
>
> **Ce que vous apprenez ici :** *Comment construire une mesure reproductible plutôt que de juger à l'œil. Pourquoi une variable à la fois. Pourquoi la solution attendue n'est pas toujours celle que la mesure retient. Et où se trouve réellement la marge de progrès d'un RAG une fois le matériel en place.*

Annexe : commandes de référence rapide

Ollama sur l'hôte

Commande	Description
ollama pull qwen3:14b	Télécharger le modèle principal
ollama pull qwen3:4b	Télécharger le modèle juge
ollama pull nomic-embed-text	Télécharger le modèle d'embedding
ollama list	Lister les modèles disponibles
ollama ps	Modèles chargés, placement et fenêtre de contexte
ollama stop <modèle>	Décharger un modèle sans arrêter le service
nvidia-smi	Carte, mémoire occupée, température, puissance

Docker Compose dans la VM

Commande	Description
docker compose up -d	Démarrer les services
docker compose up -d --build rag-api	Reconstruire et redémarrer la RAG API
docker compose logs -f rag-api	Suivre les journaux de la RAG API
docker compose ps	État des services
docker exec -w /app rag-api python3 -c "import main; print(main.LLM_MODEL)"	Lire un réglage tel qu'il est réellement appliqué

Vector store et contrôles

Commande	Description
http://localhost:6333/dashboard	Interface web du vector store
curl -X POST .../collections/documents/snapshots	Créer un instantané de sauvegarde
POST /admin/sync	Lancer la synchronisation complète du corpus
test_cloisonnement.py <cas.json>	Contrôle du cloisonnement, mode accès

Commande	Description
test_cloisonnement.py <cas.json> --mode complet	Contrôle de bout en bout sur un cas
mesure_s10.py <questions.json>	Campagne de mesure sur le jeu de questions

Ressources

Ressource	URL
Guide de déploiement, §0 à §17	doit4everyone.github.io/ia-locale-pme-suisse/docs/stack-ia-locale/
Scripts Python anonymisés	doit4everyone.github.io/ia-locale-pme-suisse/scripts/stack-ia-locale/
Guide décisionnel IA locale pour PME suisse	doit4everyone.github.io/ia-locale-pme-suisse/guides/
Qdrant	qdrant.tech/documentation
Ollama	ollama.com/docs
Open WebUI	docs.openwebui.com
vLLM	docs.vllm.ai

> *Document produit par DoIt4Everyone, octobre 2026. Phases 1 à 12 validées en lab sur VM-RAG-LAB (Ubuntu Server 26.04 LTS), d'abord sur CPU. Phases 13 à 15 validées sur un tenant Microsoft 365 de test. Phases 16 et 17 validées avec une carte NVIDIA RTX 5060 Ti 16 Go. Les versions des bibliothèques évoluent rapidement : toujours vérifier la compatibilité dans la documentation officielle avant installation.*
>
> *Rédigé à titre documentaire, sans dépendance à aucun constructeur, éditeur ou intégrateur cité. Références, structuration et aide à la rédaction assistées par IA, avec validation humaine finale.*